

Verity Lens

System do ostrożnej weryfikacji informacji,
wiarygodności newsów, zrzutów ekranu i ryzyka
obrazów AI

Autor: *do uzupełnienia*

Kierunek / grupa: *do uzupełnienia*

Prowadzący: *do uzupełnienia*

29 maja 2026

Contents

Streszczenie	3
1 Wprowadzenie	3
2 Zakres i wymagania	3
2.1 Wymagania funkcjonalne	3
2.2 Wymagania niefunkcjonalne	4
3 Architektura systemu	4
3.1 Przepływ danych	4
4 Metoda działania	5
4.1 Fakty i twierdzenia	5
4.2 News credibility	5
4.3 Screenshot OCR	5
4.4 AI image risk	5
5 Implementacja	5
6 Refaktoryzacja	6
7 Wyniki testów	6
7.1 Testy automatyczne	6
7.2 Acceptance benchmark	6
8 Wdrożenie na PC i telefon	7
8.1 PC	7
8.2 Telefon	7
8.3 Bramka wydania	8
9 Ograniczenia i ryzyka	10
10 Wnioski	10
A Najważniejsze komendy	10
B Kontrakt API	11

Streszczenie

Niniejsze sprawozdanie opisuje projekt *Verity Lens*, aplikację do lokalnej i chmurowej weryfikacji informacji. System analizuje cztery typy wejścia: tekstowe twierdzenia, adresy URL artykułów, zrzuty ekranu oraz obrazy badane pod kątem ryzyka wygenerowania przez AI. Najważniejszym założeniem projektowym jest ostrożność decyzyjna: aplikacja zwraca werdykt **true** lub **fake** tylko wtedy, gdy ma silny sygnał źródłowy lub stabilną regułę wiedzy, a w pozostałych przypadkach zwraca **uncertain**.

Dokument przygotowano zgodnie z typową strukturą raportu inżynierskiego: cel, tło, metoda, implementacja, wyniki, dyskusja, wnioski i załączniki. Taką organizację zalecają między innymi materiały KU Leuven dotyczące raportów technicznych, wytyczne IEEE Professional Communication Society oraz akademickie wskazówki Cornell Engineering dla raportów badawczych [1, 2, 3].

1 Wprowadzenie

Fałszywe lub mylące informacje w internecie pojawiają się w różnych formach: jako krótkie twierdzenia, artykuły informacyjne, posty w mediach społecznościowych, zrzuty ekranu oraz obrazy. Klasyczny klasyfikator *fake/real* jest niewystarczający, ponieważ często nie posiada dostępu do jednoznacznych dowodów. W projekcie przyjęto więc podejście produktowe: system ma pomagać użytkownikowi ocenić wiarygodność informacji, a nie udawać absolutną pewność.

Główne cele systemu:

- sprawdzanie prostych faktów i twierdzeń tekstowych;
- ocena wiarygodności artykułów informacyjnych na podstawie źródła, jakości artykułu oraz niezależnego potwierdzenia;
- analiza zrzutów ekranu przez OCR i przekazanie odczytanego tekstu do tego samego silnika weryfikacji;
- konserwatywna ocena ryzyka, że obraz został wygenerowany przez AI;
- udostępnienie produktu na komputerze, w przeglądarce, jako aplikacja desktopowa Windows oraz jako klient mobilny Flutter.

2 Zakres i wymagania

2.1 Wymagania funkcjonalne

Moduł	Wymaganie
Facts	Użytkownik wpisuje twierdzenie; system zwraca true , fake albo uncertain wraz z krótkim uzasadnieniem.
News	Użytkownik podaje URL artykułu; system zwraca ocenę credibility , a nie wymuszony werdykt prawda/fałsz.
Screenshots	Użytkownik przesyła obraz; OCR odczytuje tekst, a wynik przechodzi przez kanoniczny pipeline.

Images	Użytkownik przesyła zdjęcie/obraz; system zwraca etykietę ryzyka AI: <code>likely_ai</code> , <code>likely_not_ai</code> albo <code>uncertain</code> .
Mobile	Aplikacja Flutter działa jako klient API i może pracować poza komputerem użytkownika po wdrożeniu backendu do chmury.

2.2 Wymagania niefunkcjonalne

- stabilny publiczny kontrakt JSON dla API;
- ostrożność: brak twardego werdyktu bez silnego sygnału;
- możliwość uruchomienia lokalnego i chmurowego;
- testowalność przez unit tests, testy Flutter i acceptance benchmark;
- brak ciężkich zależności ML w minimalnym obrazie chmurowym.

3 Architektura systemu

System składa się z jednego kanonicznego silnika oraz kilku interfejsów użytkownika. Silnik znajduje się w katalogu `src/factcheck/`. Interfejsy wykorzystują ten sam pipeline, dzięki czemu CLI, API, Web UI, desktop i telefon nie implementują osobnych decyzji.

Warstwa	Pliki	Rola
Core engine	<code>src/factcheck/service.py</code>	orkiestracja pipeline'u
Decision layer	<code>src/factcheck/decision.py</code>	werdykt i ostrożność decyzyjna
Retrieval	<code>src/factcheck/retrieval.py</code>	wyszukiwanie i dobór dokumentów
News credibility	<code>src/factcheck/news_credibility.py</code>	scoring artykułów newsowych
Image/OCR	<code>src/factcheck/image_analysis.py</code>	OCR i analiza ryzyka AI
REST API	<code>src/api_factcheck.py</code>	endpointy <code>/factcheck</code> i <code>/factcheck-image</code>
Web/Desktop UI	<code>src/ui.py</code> , <code>src/desktop_app.py</code>	przeglądarka i wrapper PyWebView
Mobile	<code>apps/fake_news_detector_flutter/</code>	klient Android/iOS/Web

3.1 Przepływ danych

1. Wejście użytkownika trafia do CLI, REST API, Web UI albo aplikacji Flutter.
2. Backend normalizuje tekst lub pobiera treść URL.
3. Orkiestrator wybiera wewnętrzną strategię: fakt jawnie sprawdzony, news credibility, wiedza lokalna, OCR lub analiza obrazu.
4. Wynik jest zwracany w jednym kontrakcie JSON. Najważniejsze pola to `verdict`, `confidence`, `summary`, `evidence`, `trace`, `credibility` oraz `image_analysis`.

4 Metoda działania

4.1 Fakty i twierdzenia

Dla krótkich stabilnych faktów system używa lokalnych reguł, arytmetyki i wybranych źródeł wiedzy. Dla twierdzeń wymagających źródeł pipeline najpierw szuka stron fact-checkingowych z jawnym werdyktem, a następnie może użyć zaufanych źródeł informacyjnych jako fallback.

4.2 News credibility

Zwykły artykuł informacyjny nie dostaje automatycznie werdyktu **true** lub **fake**. Zamiast tego wyliczane są:

- `source_score` – zaufanie do domeny;
- `article_quality_score` – obecność tytułu, daty, autora, treści i cech artykułu;
- `corroboration_score` – liczba niezależnych zaufanych źródeł opisujących ten sam temat;
- `risk_score` – flagi ryzyka, np. brak tekstu, źródło społecznościowe lub sensacyjny język.

4.3 Screenshot OCR

Zrzut ekranu jest walidowany, odczytywany przez OCR, a następnie tekst jest przekazywany do głównego silnika. Jeżeli użytkownik poda jednoznaczne twierdzenie w polu pytania, to ono może zastąpić zaszumiony tekst OCR jako główny przedmiot weryfikacji.

4.4 AI image risk

Moduł obrazu nie udowadnia autentyczności ani fałszu zdjęcia. Wykorzystuje metadane, sygnały forensyczne i opcjonalny klasyfikator obrazu. W środowisku chmurowym i w paczce desktopowej domyślnie używany jest tryb `metadata_only`, aby uniknąć ciężkich modeli i fałszywych oskarżeń.

5 Implementacja

Implementacja backendu jest oparta na Pythonie i FastAPI. Główny kontrakt publiczny znajduje się w `src/api_factcheck.py`. Udostępnia endpointy `/health`, `/ready`, `/factcheck` oraz `/factcheck-image`. Interfejs webowy w `src/ui.py` korzysta z tych samych struktur wynikowych co API. Wrapper desktopowy `src/desktop_app.py` uruchamia lokalny serwer i otwiera go w oknie aplikacji.

Aplikacja mobilna Flutter znajduje się w `apps/fake_news_detector_flutter/`. Nie zawiera osobnego silnika decyzyjnego; wysyła zapytania do backendu przez klasę klienta API. Dzięki temu PC, web, desktop i telefon korzystają z tych samych reguł i tego samego kontraktu JSON.

6 Refaktoryzacja

W pierwszym etapie refaktoryzacji usunięto konflikt dwóch API. Stary plik `src/api.py` ładował prototypowy, niekanoniczny model Torch/DistilBERT przy imporcie. Został zastąpiony lekkim endpointem kompatybilności `/predict`, który deleguje do aktualnego silnika `src.api_factcheck`. Zmiana zmniejsza ryzyko, że użytkownik uruchomi nieaktualny produkt. Analogicznie `src/predict.py` nie uruchamia już starego modelu multimodalnego, tylko zachowuje kompatybilne argumenty i deleguje do kanonicznego silnika.

Kod treningowy i multimodalny został oddzielony od ścieżki produktu. Najważniejsza zasada dalszego porządkowania: kod produkcyjny ma prowadzić do jednego mózgu, czyli `src/factcheck/`. Stare pliki treningowe, modelowe i kompatybilnościowe przeniesiono z `src/` do `archive/legacy_multimodal/` oraz `archive/legacy_factcheck_v1/`. Lista plików archiwalnych została opisana w `docs/legacy_ml_inventory.md`, aby oddzielić prototyp badawczy od ścieżki produktu. Dodatkowo domyślne pliki zależności `requirements.txt` oraz `requirements.lock.txt` zostały ograniczone do lekkiego runtime'u produktu. Zależności opcjonalnego modelu AI przeniesiono do `requirements.optional-ai.txt`, a zależności archiwalnego treningu do `requirements.legacy-train.txt`.

7 Wyniki testów

7.1 Testy automatyczne

Aktualny stan po ostatnim uruchomieniu bramki wydania:

Zakres	Komenda	Wynik
Backend tests	<code>unittest</code>	113 testów OK
Legacy API	<code>test_legacy_api</code>	OK
Product gate	<code>acceptance</code>	93/93 OK; każda sekcja spełnia próg 95%
Quality pack live	<code>quality_pack_v3</code>	17/17 OK; 0 failures, 0 errors
Release gate	<code>release_gate</code>	OK; 20 kroków, bez debug skipów
Flutter analyze	<code>flutteranalyze</code>	No issues found
Flutter tests	<code>fluttertest</code>	11/11 OK

7.2 Acceptance benchmark

Dodano skrypt `scripts/run_product_acceptance.py`. Sprawdza on cztery sekcje produktu: Facts, News, Screenshots i Images, a także kontrakt API używany przez aplikację mobilną. Wynik ostatniego uruchomienia:

Sekcja	Poprawne	Wszystkie	Skuteczność
Facts	44	44	100%
News	15	15	100%
Screenshots	12	12	100%
Images	11	11	100%
API/mobile contract	11	11	100%

Ten wynik nie oznacza jeszcze gwarancji 95% na całym internecie. Oznacza natomiast, że powstała mierzalna bramka regresji produktu: każda sekcja ma własny minimalny

próg liczby przypadków (Facts 40, News 15, Screenshots 12, Images 11, API/mobile 11) i minimum 95% skuteczności. Bramkę można dalej rozszerzać o przypadki live i trudne przykłady, aż próba będzie wystarczająco reprezentatywna dla końcowej prezentacji.

Dodatkowo uruchomiono live quality pack z aktualnych źródeł RSS oraz ręcznych przypadków granicznych. Ostatni wynik to 17/17 przypadków OK, 0 błędów i 0 awarii wykonania; pełne dane zapisano w `reports/quality_pack_v3_live.md` oraz `reports/quality_pack_v3_live.json`.

8 Wdrożenie na PC i telefon

8.1 PC

Produkt działa lokalnie jako:

- REST API: `python -m uvicorn src.api_factcheck:app -host 0.0.0.0 -port 8001;`
- Web UI: `python -m uvicorn src.ui:app -host 127.0.0.1 -port 8002;`
- aplikacja desktopowa: `run_desktop_app.bat`.

Skrypt `scripts/build_windows.ps1` tworzy folder desktopowy oraz przenośną paczkę ZIP. Paczka desktopowa nie zawiera opcjonalnych zależności Torch/Transformers. Skrypt `scripts/verify_desktop_package.ps1` sprawdza obecność pliku EXE w ZIP, brak ciężkich pakietów ML oraz uruchamia packaged smoke test.

8.2 Telefon

Aplikacja Flutter jest klientem API, więc telefon wymaga publicznego backendu HTTPS. Projekt zawiera konfigurację Render oraz lekki zestaw zależności API. Skrypt budowania Render tworzy paczkę deploymentu w katalogu wynikowym dla chmury. Plik `dockerignore` wyklucza z obrazu archiwalny prototyp ML i artefakty lokalne. Klient mobilny pokazuje baner statusu API, który sprawdza endpoint `/health` i informuje użytkownika, czy telefon realnie widzi backend. Build Android używa identyfikatora pakietu `app.veritylens.mobile`. Do demonstracji na fizycznym telefonie w tej samej sieci Wi-Fi dodano skrypt budowania release APK dla sieci LAN. Tworzy on APK oraz plik statusu połączenia z adresem API, trybem, rozmiarem i SHA256 w katalogu wynikowym dla telefonu. Do tymczasowej demonstracji przez internet bez stałego wdrożenia dodano także skrypt `LocalTunnel`. Tworzy on publiczny adres HTTPS, sprawdza `/health`, `/ready` i próbny fact-check, a następnie buduje `VerityLens-internet.apk`. Ten tryb działa tylko wtedy, gdy komputer i proces tunelu pozostają uruchomione; ścieżką docelową dla telefonu pozostaje Render. Przed instalacją APK można zweryfikować przez osobny skrypt, który zapisuje rozmiar pliku, skrót SHA256, adres API oraz wyniki probe. Skrypt sprawdza również zawartość APK i potwierdza, że oczekiwany adres API jest osadzony w binarnym pliku Flutter, więc stary build nie zostanie przypadkowo opisany jako aktualny. Dla obecnych artefaktów raporty `PHONE_APK_VERIFICATION.json` oraz `PHONE_LAN_APK_VERIFICATION.json` potwierdzają osobno APK internetowe i APK dla tej samej sieci Wi-Fi. Dodatkowo strona instalacyjna w katalogu `outputs/phone_download/` pozwala pobrać APK z telefonu przez lokalny

adres HTTP. Raport gotowości telefonu sprawdza także, czy ta strona odpowiada kodem HTTP 200, czy pobieranie APK zwraca oczekiwany rozmiar pliku oraz czy raporty APK potwierdzają tryb **release**. Osobny link do stałego cloud APK jest pokazywany na stronie instalacyjnej dopiero wtedy, gdy `PHONE_CLOUD_APK_VERIFICATION.json` potwierdza tryb **release**, aktualny stamp źródeł Flutter, osadzony adres Render API oraz działający probe API, a `phone_readiness.json` ma `phone_permanent_cloud=true`. Jeżeli dostępny jest Android Virtual Device, skrypt emulatora instaluje i uruchamia APK, a wynik zapisuje w `PHONE_EMULATOR_SMOKE.json`. Dowód z fizycznego telefonu jest oddzielny: `PHONE_DEVICE_SMOKE.json` może być raportem pominiętym bez autoryzowanego urządzenia, natomiast finalne potwierdzenie wymaga uruchomienia z opcjami `-RequireDevice -RequirePhysicalDevice`, aby emulator nie mógł zastąpić telefonu. Gdy urządzenie jest dostępne, smoke test zapisuje także `PHONE_DEVICE_SCREENSHOT.png`, SHA256 tego zrzutu ekranu, potwierdzenie sygnatury PNG z dodatkimi wymiarami oraz pola identyfikacji ADB, takie jak producent, model, wersja Androida, SDK, hardware i flaga `ro.kernel.qemu`. Ścieżka stałej chmury korzysta ze wspólnej polityki adresów URL w `scripts/cloud_url_policy.ps1`: akceptowany jest tylko prawdziwy publiczny adres HTTPS, a adresy `localhost`, prywatne IP sieci LAN, nazwy `.local`, jednowyrazowe hosty i tymczasowe tunele nie mogą zostać uznane za gotowość telefonu bez komputera. Po wdrożeniu backendu na Render można zbudować APK:

```
.\scripts\build_phone_for_cloud.ps1 '  
-ApiBaseUrl https://app.onrender.com '  
-Mode release
```

8.3 Bramka wydania

Do projektu dodano jedną komendę kontroli przed demonstracją lub oddaniem:

```
.\scripts\run_release_gate.ps1  
.\scripts\run_release_gate.ps1 '  
-ApiBaseUrl https://app.onrender.com '  
-BuildCloudApk -CloudApkMode release  
.\scripts\check_final_external_prereqs.ps1 '  
-ApiBaseUrl https://app.onrender.com '  
-RequireReady  
.\scripts\finalize_cloud_phone_submission.ps1 '  
-ApiBaseUrl https://app.onrender.com
```

Pierwsza wersja wykonuje 20 kroków: sprawdza składnię Pythona, testy backendu, product acceptance, budowę PDF raportu, konfigurację deployu Render, przygotowanie paczki Render, smoke test rozpakowanej paczki Render, raport statusu chmury, weryfikację APK internetowego, stronę instalacyjną telefonu, lokalny serwer API dla trybu LAN, weryfikację APK LAN, raport gotowości telefonu, lokalny desktop smoke, packaged desktop verification, `flutter analyze`, `flutter test`, dostępność device smoke oraz końcowe odświeżenie readiness. Druga wersja po wdrożeniu Render dodatkowo sprawdza publiczne API HTTPS, buduje APK w trybie **release** wskazujące na ten backend i weryfikuje, że adres Render jest rzeczywiście osadzony w binarnym pliku APK. Weryfikacja APK porównuje również SHA256 źródeł Flutter zapisany podczas budowy z aktualnym drzewem źródeł, aby stary APK nie przeszedł po zmianie aplikacji mobilnej. Raport `CLOUD_DEPLOYMENT_STATUS.json` zapisuje także JSON weryfikacji cloud

APK oraz sidecar `VerityLens-cloud-flutter-source-stamp.json`, aby stała paczka telefoniczna była powiązana z aktualnymi źródłami mobilnymi. Pola `verification_ok` oraz `readiness_ok` w tym raporcie są wyliczane z rzeczywistego JSON weryfikacji, trybu `release`, osadzonego adresu Render, probe API i zgodności stampu źródeł, więc parametr wrappera nie może samodzielnie oznaczyć starego lub debugowego APK jako gotowego. Gate uruchamia również lokalny smoke test paczki Render: rozpakowuje ZIP, startuje API z rozpakowanej kopii i sprawdza `/health`, `/ready` oraz próbny fact-check. Końcowy preflight chmury wykonuje ostrzejszą wersję tego testu w świeżym środowisku wirtualnym, instalując wyłącznie zależności z `requirements.api.txt` znajdującego się w rozpakowanej paczce. Przed długim wrapperem końcowym należy uruchomić `check_final_external_prereqs.ps1` z opcją `-RequireReady`; skrypt sprawdza politykę publicznego adresu Render, ponawia próby dla endpointów `/health`, `/ready` i próbnego `/factcheck` oraz sprawdza autoryzację fizycznego urządzenia przez ADB. Ten raport zapisuje również dokładną komendę `adb devices` i jej surowy wynik, aby rozróżnić brak telefonu, stany `offline/unauthorized`, sam emulator albo błąd uprawnień do uruchomienia ADB. Sekcja `Next Actions` jest zależna od wykrytego stanu, więc wskazuje akceptację komunikatu RSA, ponowne podłączenie lub restart ADB, użycie prawdziwego telefonu zamiast emulatora albo podłączenie telefonu przy pustej liście urządzeń. Procedura telefonu wymaga włączenia `USB debugging`, akceptacji komunikatu RSA i wyniku `adb devices` ze stanem `device`; stany `offline` oraz `unauthorized` nie są dowodem. Gdy podłączono więcej niż jedno urządzenie, finalny preflight i wrapper przyjmują wybrany numer seryjny przez `-PhoneDeviceId`. Jeżeli ścieżka `PATH` wskazuje niewłaściwe SDK Androida, te same skrypty oraz bramka wydania przyjmują jawny plik ADB przez `-AdbPath C:\path\to\adb.exe`. Finalny wrapper dla chmury i telefonu uruchamia gate z `-RequirePhoneDevice -RequirePhysicalPhoneDevice`, buduje cloud APK, wymaga zrzutu ekranu z fizycznego Androida, odrzuca emulator jako dowód końcowy, zapisuje raport `final_cloud_phone_submission_latest.*`, traktuje pozytywny raport jako niezmienny, odświeża paczkę oddania tak, aby ten sam raport był spakowany pod `final/`, i natychmiast uruchamia weryfikator tej paczki. Nieudane raporty finalizera pozostają w katalogu `reports/` do diagnostyki, ale normalna paczka oddania kopiuje raport końcowy tylko wtedy, gdy ma `ok=true`; weryfikator zapisuje status `final_cloud_phone_submission_report.status`. Ponieważ raport końcowy znajduje się wewnątrz ZIP, pola SHA dotyczące `submission_zip` i JSON weryfikatora w tym raporcie są oznaczone jako `pre_final_report_bundle_snapshot`; dowodem końcowego ZIP jest wynik weryfikatora zapisany po spakowaniu raportu. Końcowy audit oraz wrapper wymagają także efektywnego statusu chmury: `CLOUD_DEPLOYMENT_STATUS.json` musi mieć `outcome=permanent_cloud_phone_ready`, `verification_ok=true`, `readiness_ok=true` oraz `cloud_apk_verification_status.ready_to_publish=true`. Wrapper zapisuje i sprawdza również szczegóły dowodu z fizycznego telefonu z `PHONE_DEVICE_SMOKE.json`: `require_physical_device=true`, wybrany numer seryjny urządzenia, `selected_device_is_emulator=false`, zapisane pola tożsamości urządzenia, poprawny plik PNG z dodatkimi wymiarami oraz SHA256 zrzutu ekranu zgodny z plikiem `PHONE_DEVICE_SCREENSHOT.png`. Ten wrapper akceptuje tylko tryb `-CloudApkMode release`; tryb debug pozostaje dostępny wyłącznie przez bezpośrednie uruchomienie `run_release_gate.ps1` do diagnostyki lokalnej. Weryfikator paczki oddania wymaga obecności konkretnych kroków release gate, zgodności SHA256 i rozmiarów artefaktów, spójności raportów telefonu oraz czystości źródeł. Paczka oddania zawiera również czysty ZIP źródeł `source/verity-lens-source.zip`, który obejmuje kod,

testy, skrypty, konfigurację i klienta Flutter, ale wyklucza środowisko `.venv`, katalogi `build/dist/outputs`, `cache`, pliki lokalne IDE, wygenerowane rejestratory Flutter oraz archiwalne prototypy. Ostatnia weryfikacja źródeł wykazała 240 wpisów i 0 zabronionych plików generowanych lub lokalnych.

9 Ograniczenia i ryzyka

- Nie da się uczciwie zagwarantować 95% poprawności dla wszystkich możliwych newsów bez dużego, wersjonowanego benchmarku.
- Aktualne fakty polityczne i urzędowe mogą się dezaktualizować.
- OCR może błędnie odczytać zrzuty ekranu niskiej jakości.
- Analiza AI obrazów jest oceną ryzyka, nie dowodem.
- Render Free może zasypiać, powodując wolny pierwszy request.
- Telefon nie działa offline, ponieważ silnik Python działa po stronie backendu.
- Stała gotowość telefonu bez komputera wymaga jeszcze prawdziwego wdrożenia Render HTTPS i APK zbudowanego pod ten adres.
- Finalny dowód fizycznego Androida wymaga autoryzowanego urządzenia USB i smoke testu z `-RequireDevice -RequirePhysicalDevice`; emulator jest tylko dowodem platformowym.

10 Wnioski

Projekt ma działający rdzeń, interfejs webowy, desktopowy i mobilny. Lokalna ścieżka PC, paczka desktopowa, Render bundle, APK internetowe, APK LAN, emulator i paczka oddania są objęte release gate oraz verifierem. Najważniejszym krokiem produktowym jest teraz utrzymanie i dalsze rozszerzanie acceptance benchmarku, stałe wdrożenie backendu do chmury oraz końcowy smoke test na fizycznym telefonie, aby aplikacja mobilna działała bez komputera i miała pełny dowód urządzeniowy. Najważniejszym krokiem inżynierskim jest dalsze utrzymywanie jednego jasnego runtime produktu i trzymanie kodu badawczego w archiwum.

A Najważniejsze komendy

```
python -m unittest discover -s tests -v
python scripts/run_product_acceptance.py
.\scripts\run_release_gate.ps1
.\scripts\check_phone_readiness.ps1
.\scripts\verify_phone_apk.ps1 -ExpectedMode release -RequireHttps
.\scripts\prepare_phone_install_page.ps1 -StartServer
.\scripts\smoke_phone_emulator.ps1 '
    -AvdName FakeNewsDetector_API36 -RequireEmulator
.\scripts\smoke_phone_on_device.ps1 -RequireDevice '
```

```

    -RequirePhysicalDevice
adb devices
.\scripts\check_final_external_prereqs.ps1 '
    -ApiBaseUrl https://app.onrender.com '
    -RequireReady
.\scripts\finalize_cloud_phone_submission.ps1 '
    -ApiBaseUrl https://app.onrender.com
.\scripts\make_submission_bundle.ps1
.\scripts\verify_submission_bundle.ps1
python -m uvicorn src.api_factcheck:app --port 8001
python -m uvicorn src.ui:app --port 8002
flutter analyze
flutter test

```

B Kontrakt API

```

GET /health
GET /ready
POST /factcheck
POST /factcheck-image

```

References

- [1] KU Leuven Faculty of Engineering Science. *Report writing: structure and content*. <https://eng.kuleuven.be/en/study/engineering-essentials/reporting/structure-content>
- [2] IEEE Professional Communication Society. *Write Effective Reports*. <https://procomm.ieee.org/communication-resources-for-engineers/written-reports/write-effective-reports/>
- [3] Cornell Duffield Engineering. *Undergraduate Research Report Guidelines*. <https://www.duffield.cornell.edu/undergraduate-research-report-guidelines/>
- [4] Delaney, D. and Brown, S. *Guidelines for Documents Produced by Student Projects in Software Engineering*. Department of Computer Science, National University of Ireland, Maynooth, 2006. <https://mural.maynoothuniversity.ie/id/eprint/5971/>